

Nombre y Apellido:..... Curso: Profesor:

TEORÍA - Single Choice								TEORÍA - Desarrollo			PRÁCTICA				
1	2	3	4	5	6	7	8	1	2	3	1	2	3	4	5

Teoría - Single choice: elija para cada pregunta la opción correcta (solo una es válida)

- 1- Considerando que las **interrupciones** no están deshabilitadas, las mismas se **atienden**:
a) Al finalizar de ejecutar la instrucción en curso b) En cuanto ocurren
c) Cuando el planificador elija a otro proceso a ejecutar d) Luego de desalojar al proceso actual
- 2- ¿Cuál de las siguientes es una interrupción **sincrónica**?
a) Fin de quantum b) Fin de Entrada/Salida c) Error en un dispositivo d) División por cero
- 3- Comparado con utilizar múltiples KLTs, utilizar **múltiples procesos** cooperativos permite:
a) Ejecutar con mayor paralelismo b) Mayor eficiencia en la comunicación
c) Cambios de contexto más rápidos d) Ninguna es correcta
- 4- ¿Cuál de las siguientes afirmaciones es **falsa sobre FIFO**?
a) Podría permitir que un proceso monopolice la CPU b) Minimiza los cambios de contexto
c) Podría ser útil para correr procesos secuenciales d) Perjudica a los procesos CPU bound
- 5- ¿Sobre cuál de las siguientes **transiciones** actúa el planificador de **mediano plazo**?
a) susp/blocked -> blocked b) ready -> susp/ready c) susp/ready -> ready d) Todas son correctas
- 6- ¿Cuál de los siguientes **no es un requerimiento** para una buena **solución a la condición de carrera**?
a) No desalojo b) Progreso c) Espera limitada d) Mutua exclusión
- 7- El mecanismo de **evasión de deadlocks**:
a) Tiene una alta flexibilidad de asignación de recursos b) Tiene overhead bajo según con qué frecuencia ejecute
c) Requiere que los procesos indiquen sus peticiones máximas d) Todas son correctas
- 8- **Comparando** deadlocks y livelocks:
a) El livelock es generalmente más difícil de detectar b) El deadlock retiene los recursos mientras que el livelock no
c) El deadlock es generalmente más difícil de detectar d) Ninguna es correcta

Teoría - Desarrollar: responda las siguientes preguntas justificando sus respuestas

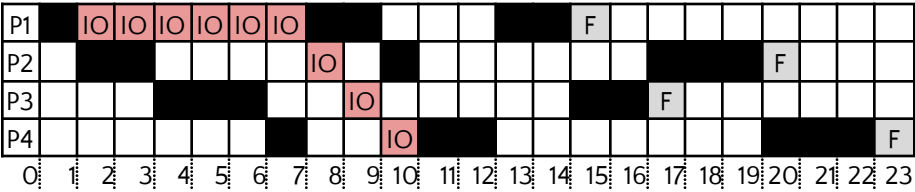
- 1 - Dé un **ejemplo en pseudocódigo** donde se acceden/utilizan el stack, el heap y la sección de datos del proceso. Indique dónde se accede/utiliza cada uno en el código que escribió.
- 2 - **Compare** los algoritmos Virtual Round Robin, Round Robin y FIFO usando los siguientes criterios: atención de procesos I/O bound, inanición y tiempo de respuesta promedio.
- 3 - **Indique** qué medidas puede tomar el Sistema Operativo al detectar un deadlock. **Explique** al menos una ventaja y una desventaja por cada una de estas medidas.

Práctica - Desarrollar:
 resuelva los siguientes ejercicios justificando sus respuestas

1 - Se dispone de un sistema monoprocesador cuyo planificador a corto plazo utiliza un algoritmo SRT (SJF con desalojo) el cual realiza sus estimaciones con la fórmula de media exponencial $Est_{N+1} = (Est_N * \alpha) + (CPU_N * (1 - \alpha))$ con un valor de $\alpha=0,5$. A partir de la siguiente traza de ejecución, **elabore el diagrama de Gantt** correspondiente sabiendo que todos los procesos están inicialmente listos en orden numérico ascendente:

	Est. anterior	CPU anterior	CPU	E/S	CPU
P1	2	3	2	1	2
P2	2	1	6	1	3
P3	5	2	1	3	1
P4	8	1	3	1	2

2 - Dado el siguiente diagrama GANTT, donde se sabe que los procesos están inicialmente listos en orden numérico ascendente:



Responda ¿Qué algoritmo de planificación de corto plazo está utilizando el Sistema Operativo?

3 - Una maratón tiene un sistema de tickets para inscribirse a distintas carreras según la experiencia de cada corredor. Los mismos utilizan una única terminal donde escriben en qué carrera quieren inscribirse (hay 3 para elegir), luego de recibir el ticket esperan a ser llamados por el número del mismo. La terminal genera el ticket, agrega su número a la lista de turnos de esa carrera y le entrega el ticket al corredor. Por cada carrera, hay un empleado que va tomando números de su lista de turnos para llamarlos a posicionarse. **Sincronice el siguiente pseudocódigo** concurrente utilizando semáforos de forma que cumpla con el comportamiento descrito sin generar deadlocks ni starvation.

Corredor (100 instancias)	Terminal de Tickets (1 instancia)	Empleado (3 instancias)	struct ticket_t { idCarrera; // 0 a 2 número; // 0 a 99 }
acercarseATerminal() escribirlaCarreraEnInput() ticket_t ticket = recibirTicket() posicionarseCarrera()	while(true) { id = leerInput() ticket_t ticket = generarTicket(id) entregarTicket() agregar (ticket.número, turnos[id]) }	while(true) { numLamar = retirar (turnos[getId()]) llamarCorredor() }	//estructura del tipo de dato ticket_t

Notas: - La función **leerInput()** falla si no hay nada escrito. La función **generarTicket()** le asigna un número entre 0 y 99 al ticket.
 - El proceso Empleado dispone de una función **getId()** que devuelve el id correspondiente a su carrera.

4 - El siguiente pseudocódigo simula la interacción de N usuarios en la red social “Z” y un proceso analizador que toma cada uno de los posts generados para analizarlos. A pesar de estar sincronizado, el sistema se desempeña más lento de lo esperado y frecuentemente deja de funcionar hasta que el administrador reinicia el proceso Analizador:

Usuario (N instancias)	Analizador (1 instancia)	Variables compartidas:
While(1) { wait (mutexPosts); post = generarPost(); postear(post, postsNuevos); signal (hayPosts); mostrarEnPantalla(post); signal (mutexPosts); }	While(1) { wait (mutexPosts); wait (hayPosts); post = obtenerPost(postsNuevos); resultado = procesar(post) guardarEnDisco(resultado) signal (mutexPosts) }	postsNuevos -> cola que contiene los nuevos posts a analizar <u>Semáforos:</u> hayPosts -> semáforo contador inicializado en 0 mutexPosts -> semáforo mutex

Encuentre errores y/o mejoras en la sincronización planteada (no sincronizar nuevamente, solamente marcar y explicar los errores encontrados).

5 - Se encuentran N procesos compartiendo distintos recursos, de los cuales existe una sola instancia de cada uno, ejecutando el siguiente pseudocódigo:

```

while(true){
    t_rec recurso1 = get_resource() //devuelve aleatoriamente un recurso compartido, bloqueante si el recurso no está disponible
    t_rec recurso2 = get_resource()
    usar_recursos();
    devolver_recursos();
}
    
```

- Demuestre** que dichos procesos podrían quedar en deadlock.
- Considere las condiciones necesarias y suficientes para la existencia de deadlock y **proponga una solución para prevenirlo** sin usar semáforos.

Criterio mínimo de aprobación: 4 teóricas SC correctas, 1,5 teóricas a desarrollar correctas, 3 ejercicios correctos.